

Method: add(newEntry)

ArrayList

```
22 public boolean add(T newEntry) {  
23     array[numberOfEntries] = newEntry;  
24     numberOfEntries++;  
25     return true;  
}
```

LinkedList

```
30 public boolean add(T newEntry) {  
31     Node newNode = new Node(newEntry);  
32     if (isEmpty()) {  
33         firstNode = newNode;  
34     } else {  
35         Node currentNode = firstNode;  
36         while (currentNode.next != null) {  
37             currentNode = currentNode.next;  
38         }  
39         currentNode.next = newNode;  
40     }  
41     numberOfEntries++;  
42     return true;  
43 }
```

Method: add(newPosition, newEntry)

ArrayList

```

29 public boolean add(int newPosition, T newEntry) {
30     boolean isSuccessful = true;
31
32     if ((newPosition >= 1) && (newPosition <= numberOfEntries + 1)) {
33         makeRoom(newPosition);
34         array[newPosition - 1] = newEntry;
35         numberOfEntries++;
36     } else {
37         isSuccessful = false;
38     }
39
40     return isSuccessful;
41 }
42
129 private void makeRoom(int newPosition) {
130     int newIndex = newPosition - 1;
131     int lastIndex = numberOfEntries - 1;
132     for (int index = lastIndex; index >= newIndex; index--) {
133         array[index + 1] = array[index];
134     }
135 }

```

LinkedList

```

47 public boolean add(int newPosition, T newEntry) {
48     boolean isSuccessful = true;
49
50     if ((newPosition >= 1) && (newPosition <= numberOfEntries + 1)) {
51         Node newNode = new Node(newEntry);
52         if (isEmpty() || (newPosition == 1)) {
53             newNode.next = firstNode;
54             firstNode = newNode;
55         } else {
56             Node nodeBefore = firstNode;
57             for (int i = 1; i < newPosition - 1; ++i) {
58                 nodeBefore = nodeBefore.next;
59             }
60             newNode.next = nodeBefore.next;
61             nodeBefore.next = newNode;
62             numberOfEntries++;
63         } else {
64             isSuccessful = false;
65         }
66     }
67     return isSuccessful;
68 }

```

Method: remove(position)

ArrayList

```

44 public T remove(int givenPosition) {
45     T result = null;
46     if ((givenPosition >= 1) && (givenPosition <= numberOfEntries)) {
47         result = array[givenPosition - 1];
48         if (givenPosition < numberOfEntries) {
49             removeGap(givenPosition);
50         }
51         numberOfEntries--;
52     }
53     return result;
54 }

138 private void removeGap(int givenPosition) {
139     int removedIndex = givenPosition - 1;
140     int lastIndex = numberOfEntries - 1;
141
142     for (int index = removedIndex; index < lastIndex; index++) {
143         array[index] = array[index + 1];
144     }
145 }

```

LinkedList

```

71 public T remove(int givenPosition) {
72     T result = null;
73
74     if ((givenPosition >= 1) &&
75         (givenPosition <= numberOfEntries)) {
76         if (givenPosition == 1) {
77             result = firstNode.data;
78             firstNode = firstNode.next;
79         } else {
80             Node nodeBefore = firstNode;
81             for (int i = 1; i < givenPosition - 1; ++i) {
82                 nodeBefore = nodeBefore.next;
83             }
84             result = nodeBefore.next.data;
85             nodeBefore.next = nodeBefore.next.next;
86             numberOfEntries--;
87         }
88     }
89     return result;
90 }

```

Method: replace(givenPosition, newEntry)

ArrayList

```
62 public boolean replace(int givenPosition, T newEntry) {
63     boolean isSuccessful = true;
64     if ((givenPosition >= 1) &&
65         (givenPosition <= numberOfEntries)) {
66         array[givenPosition - 1] = newEntry;
67     } else {
68         isSuccessful = false;
69     }
70     return isSuccessful;
71 }
72 }
```

LinkedList

```
93 public boolean replace(int givenPosition, T newEntry) {
94     boolean isSuccessful = true;
95     if ((givenPosition >= 1) &&
96         (givenPosition <= numberOfEntries)) {
97         Node currentNode = firstNode;
98         for (int i = 0; i < givenPosition - 1; ++i) {
99             currentNode = currentNode.next;
100         }
101         currentNode.data = newEntry;
102     } else {
103         isSuccessful = false;
104     }
105     return isSuccessful;
106 }
107 }
```

Method: getEntry(givenPosition)

ArrayList

```
76 public T getEntry(int givenPosition) {  
77     T result = null;  
78     if ((givenPosition >= 1) &&  
79         (givenPosition <= numberOfEntries)) {  
80         result = array[givenPosition - 1];  
81     }  
82  
83     return result;  
84 }
```

LinkedList

```
111 public T getEntry(int givenPosition) {  
112     T result = null;  
113     if ((givenPosition >= 1) &&  
114         (givenPosition <= numberOfEntries)) {  
115         Node currentNode = firstNode;  
116         for (int i = 0; i < givenPosition - 1; ++i) {  
117             currentNode = currentNode.next;  
118         }  
119         result = currentNode.data;  
120     }  
121     return result;  
122 }
```

Method: contains(anEntry)

ArrayList

```
88 public boolean contains(T anEntry) {
89     boolean found = false;
90     for (int index = 0;
91         !found && (index < numberOfEntries);
92         index++) {
93         if (anEntry.equals(array[index])) {
94             found = true;
95         }
96     }
97     return found;
}
```

LinkedList

```
126 public boolean contains(T anEntry) {
127     boolean found = false;
128     Node currentNode = firstNode;
129     while (!found && (currentNode != null)) {
130         if (anEntry.equals(currentNode.data)) {
131             found = true;
132         } else {
133             currentNode = currentNode.next;
134         }
135     }
136     return found;
137 }
```

Method: getNumberOfEntries()

ArrayList

```
101 public int getNumberOfEntries() {  
102     return numberOfEntries;  
}
```

LinkedList

```
141 public int getNumberOfEntries() {  
142     return numberOfEntries;  
}
```

Method: isEmpty()

ArrayList

```
104      @Override
105      public boolean isEmpty() {
106          return numberOfEntries == 0;
107      }
```

LinkedList

```
144      @Override
145      public boolean isEmpty() {
146          boolean result;
147          result = numberOfEntries == 0;
148          return numberOfEntries == 0;
149      }
```


Method: toString()

ArrayList

```
@Override
public String toString() {
    String outputStr = "";
    for (int index = 0; index < numberOfEntries; ++index) {
        outputStr += array[index] + "\n";
    }
    return outputStr;
}
```

LinkedList

```
156
157
158
159
160
161
162
163
164
165
```



```
@Override
public String toString() {
    String outputStr = "";
    Node currentNode = firstNode;
    while (currentNode != null) {
        outputStr += currentNode.data + "\n";
        currentNode = currentNode.next;
    }
    return outputStr;
}
```

Method: add()

ArrayList

LinkedList

--	--

Method: add()	
ArrayList	LinkedList

